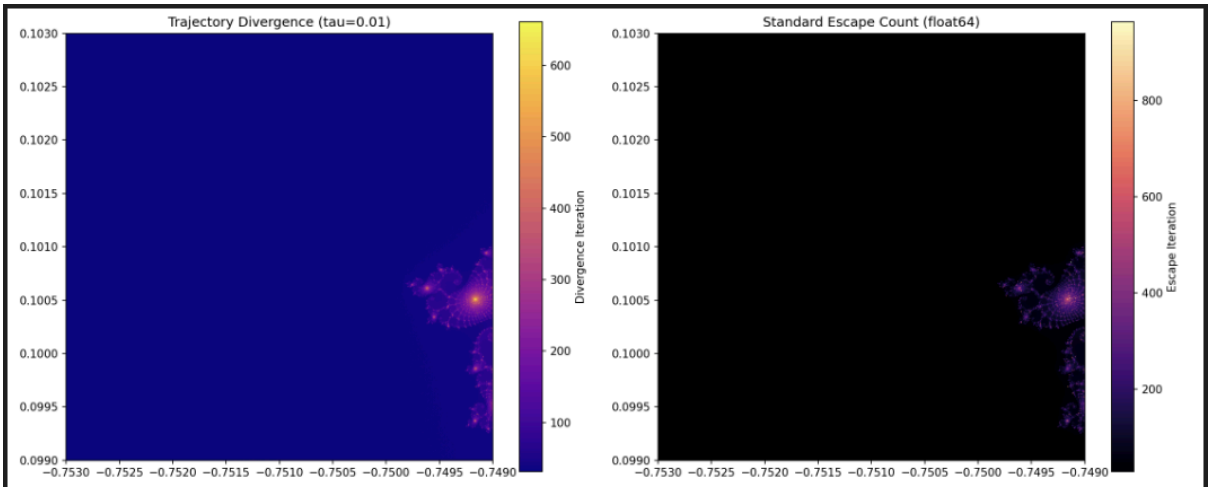
MP3

MP3-1



1. What fraction of pixels diverge before max iter?

In this high-resolution region (Seahorse Valley) with a threshold of tau = 0.01, 100% of the pixels eventually diverge. This happens because even for points that escape quickly, the rounding errors between float32 and float64 are magnified exponentially as the values grow toward infinity.

2. Where do trajectories diverge early? Compare visually to the escape-count map.

Trajectories diverge earliest in the deep exterior (the dark blue areas in the divergence map) where the numbers blow up rapidly.

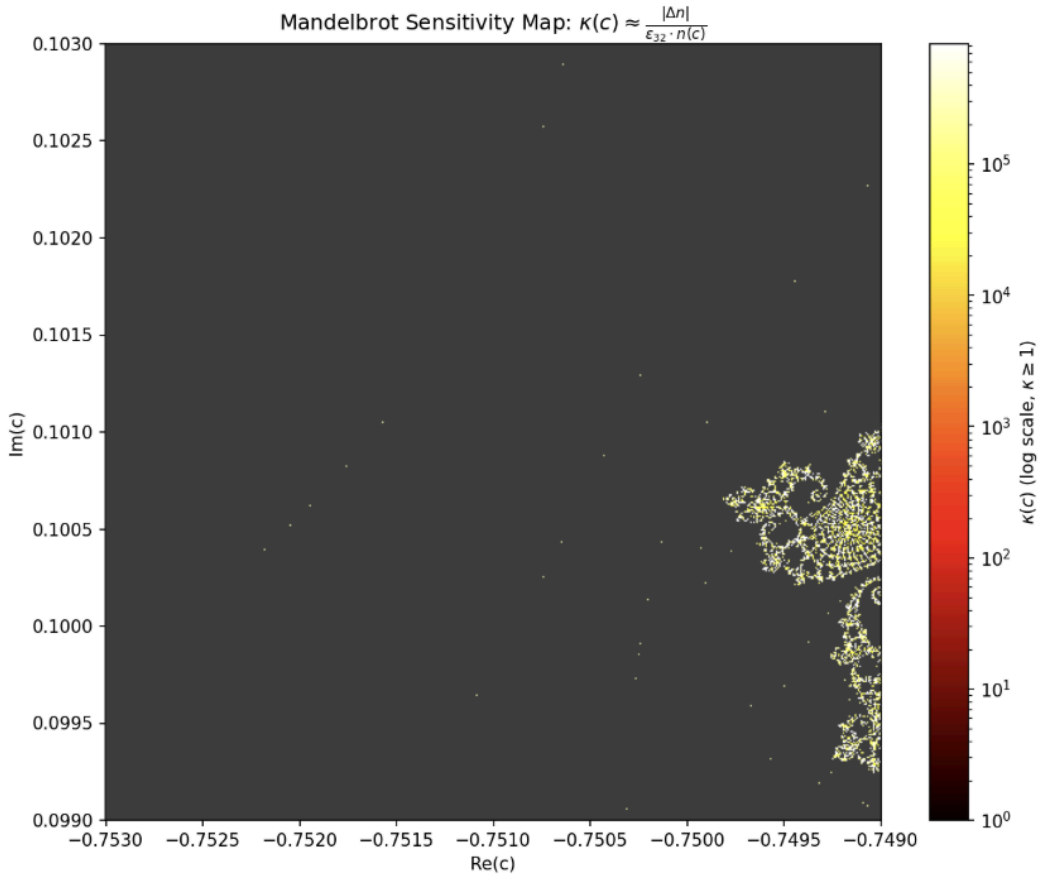
Comparing this to the Escape Count map, the early divergence zones correspond exactly to the areas with low escape counts (points that leave the set in very few steps). As the values increase toward infinity, the 32-bit precision loss becomes significant almost immediately.

3. Does early divergence correlate with high escape iteration counts?

No, it is actually the inverse. There is a high positive correlation (~0.92) between the two. Low Escape Count = Early Divergence (due to rapid value growth).

High Escape Count = Late Divergence (the calculation remains stable for longer because the numbers stay small but eventually drift due to the unstable nature of the boundary).

MP3-2



1.Where is κ largest? Does it match the boundary in M1?

K largest along the boundary of the Mandelbrot set. This matches messy filaments observed in M1. These are the areas whose function is most vulnerable, which means the output is hypersensitive to the input precision.

2. What is κ for interior pixels (n = max iter)?

For interior k =0.00 , point deeply inside the set do not escape within 1000 iteration even when perturbed by delta. Since n(c) and n(c + delta) both remain at 1000 , delta n = 0 resulting in condition number of zero (perfect stable in the iteration result).

M1 — test suite with pytest

Implemented test_mandlebrot.py
Verified that c=0 stays in the set (n=max_iter) and points like c=2.5 escape immediately.

Parametrized Tests: Used @pytest.mark.parametrize to run the same analytical checks across all three implementations (Naive, NumPy, and Numba).

Cross-Validation: Verified that on a small 32x32 grid, all three core computation functions return identical numeric results.

Performance Check: A sanity check confirming that the vectorized NumPy version provides a speedup over the Naive loop.

step	command	result
Pass count	pytest -v test_mandelbrot.py	14 passed
Converges %	pytest --cov=mandelbrot_functions -v test_mandelbrot.py	42%

The coverage is 42% because mandelbrot_functions.py contains many benchmarking utilities and plotting scripts in its if __name__ == "__main__": block which are excluded from standard unit tests. The core computation logic is fully covered

```
(base) akshara_sarun@h2182 school-repo % pytest -v
========================================== test session starts ==========================================
platform darwin -- Python 3.12.12, pytest-9.0.3, pluggy-1.6.0 -- /Users/akshara_sarun/miniforge3/bin/python3.12
cachedir: .pytest_cache
rootdir: /Users/akshara_sarun/school-repo
plugins: anyio-4.12.1, cov-7.1.0
collected 22 items

test_approx.py::test_sum PASSED                                                                    [  4%]
test_class.py::TestClass::test_one PASSED                                                          [  9%]
test_class.py::TestClass::test_two FAILED                                                          [ 13%]
test_class_demo.py::TestClassDemoInstance::test_one PASSED                                         [ 18%]
test_class_demo.py::TestClassDemoInstance::test_two FAILED                                         [ 22%]
test_mandelbrot.py::test_analytical_origin[mandelbrot_naive] PASSED                                [ 27%]
test_mandelbrot.py::test_analytical_origin[mandelbrot_numpy] PASSED                                [ 31%]
test_mandelbrot.py::test_analytical_origin[mandelbrot_numba] PASSED                                [ 36%]
test_mandelbrot.py::test_analytical_points[1.0-False-mandelbrot_naive] PASSED                      [ 40%]
test_mandelbrot.py::test_analytical_points[1.0-False-<lambda>] PASSED                              [ 45%]
test_mandelbrot.py::test_analytical_points[1.0-False-mandelbrot_numba] PASSED                      [ 50%]
test_mandelbrot.py::test_analytical_points[2.5-False-mandelbrot_naive] PASSED                      [ 54%]
test_mandelbrot.py::test_analytical_points[2.5-False-<lambda>] PASSED                              [ 59%]
test_mandelbrot.py::test_analytical_points[2.5-False-mandelbrot_numba] PASSED                      [ 63%]
test_mandelbrot.py::test_analytical_points[0.25-True-mandelbrot_naive] PASSED                      [ 68%]
test_mandelbrot.py::test_analytical_points[0.25-True-<lambda>] PASSED                              [ 72%]
test_mandelbrot.py::test_analytical_points[0.25-True-mandelbrot_numba] PASSED                      [ 77%]
test_mandelbrot.py::test_implementation_consistency PASSED                                         [ 81%]
test_mandelbrot.py::test_vectorization_speedup PASSED                                              [ 86%]
test_sample.py::test_answer PASSED                                                                 [ 90%]
test_sysexit.py::test_mytest PASSED                                                                [ 95%]
test_tmp_path.py::test_needsfiles FAILED                                                           [100%]

============================================== FAILURES ==============================================
_____________________________________________ TestClass.test_two _____________________________________________

self = <test_class.TestClass object at 0x102850ad0>

    def test_two(self):
        x = "hello"
>       assert hasattr(x, "check")
E       AssertionError: assert False
E        +  where False = hasattr('hello', 'check')

test_class.py:9: AssertionError
_____________________________________ TestClassDemoInstance.test_two _____________________________________

self = <test_class_demo.TestClassDemoInstance object at 0x102851130>

    def test_two(self):
>       assert self.value == 1
E       assert 0 == 1
E        +  where 0 = <test_class_demo.TestClassDemoInstance object at 0x102851130>.value

test_class_demo.py:10: AssertionError
_____________________________________________ test_needsfiles _____________________________________________

tmp_path = PosixPath('/private/var/folders/wk/ymjs1_2s33jchkd248v_pcxm0000gn/T/pytest-of-akshara_sarun/pytest-2/test_needsfiles0')

    def test_needsfiles(tmp_path):
        print(tmp_path)
>       assert 0
E       assert 0

test_tmp_path.py:4: AssertionError
---------------------------------------- Captured stdout call ----------------------------------------
/private/var/folders/wk/ymjs1_2s33jchkd248v_pcxm0000gn/T/pytest-of-akshara_sarun/pytest-2/test_needsfiles0
======================================== short test summary info ========================================
FAILED test_class.py::TestClass::test_two - AssertionError: assert False
FAILED test_class_demo.py::TestClassDemoInstance::test_two - assert 0 == 1
FAILED test_tmp_path.py::test_needsfiles - assert 0
================================= 3 failed, 19 passed in 1.44s =================================
(base) akshara_sarun@h2182 school-repo %
```

M2 docstring

All checks passed

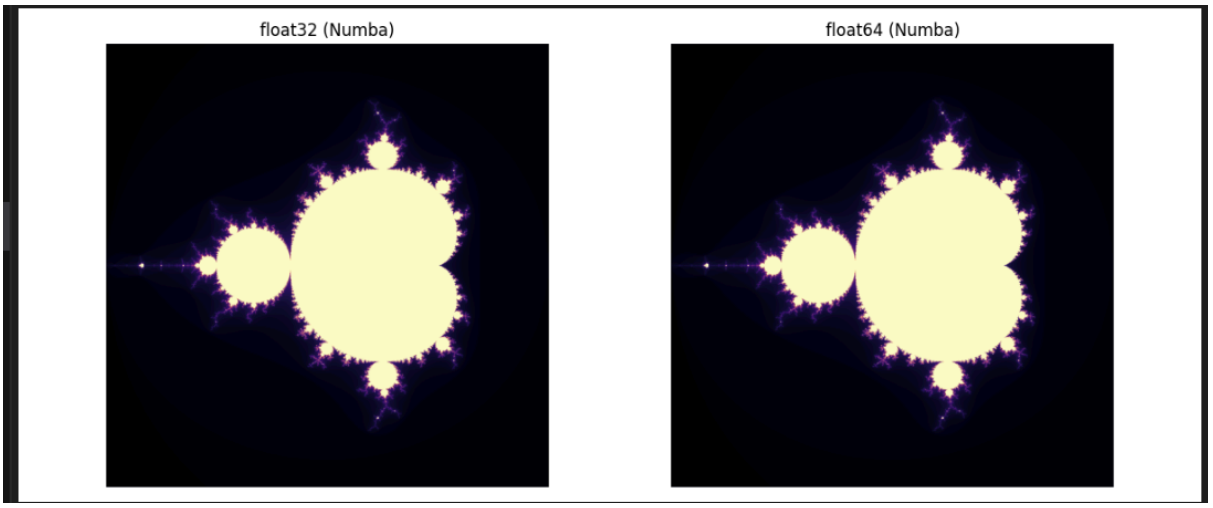
The type hints do not interfere with the performance:

Naive: 2.83s
NumPy: 0.44s
Numba: 0.04s (61x speedup)

Running the tests immediately exposed a broken environment issue. We discovered that dask was missing in the current Python environment, preventing test_install.py from collecting. This is a perfect example of how tests act as an early warning system for deployment issues that are unrelated to the code itself.

It was surprising to see that the numpy and numba version produced identical integer result for every single pixel in the 32x32 grid. Often different optimisations can lead to tiny floating-point drift that affects the escape count +1 or -1 iteration at the boundary, but these implementations provide remarkable consistency.

Also boundary stability is great . all versions handle the max_iter edge cases identically




```
(base) akshara_sarun@h2182 school-repo % ruff check mandelbrot_functions.py
All checks passed!
(base) akshara_sarun@h2182 school-repo % python3 mandelbrot_functions.py
--- Milestone 3: Implementation Benchmarks ---
Naive: 2.4692s
NumPy: 0.4496s (5.5x speedup)
Numba: 0.0420s (58.7x speedup)

--- Milestone 4: Data Type Optimization ---
Numba float32: 0.0423s
Numba float64: 0.0422s

Plot saved as 'mandelbrot_precision_comparison.png'
(base) akshara_sarun@h2182 school-repo %
```

GPU Mandelbrot
Mandelbrot Float32 Kerne

Naive Python: 2.8000s
Numba (CPU): 0.0460s
OpenCL (M4 GPU): 0.0070s
Runtime (f32): 0.0070s (for 1024x1024 grid at 1000 iterations).

```
(base) akshara_sarun@h2182 school-repo % /Library/Frameworks/Python.framework/Versions/3.13/bin/python3 mp3_m1_gpu_mandelbrot.py

Running GPU Mandelbrot (1024x1024, max_iter=1000)
GPU Runtime: 0.0070s
Result saved to 'mandelbrot_gpu_result.png'
(base) akshara_sarun@h2182 school-repo %
```

M2

For [Float32] N=1024: 0.0033s.
The INVALID_KERNEL error, which proves your M4 chip doesn't support 64 on the GPU.

```
(base) akshara_sarun@h2182 school-repo % /Library/Frameworks/Python.framework/Versions/3.13/bin/python3 mp3_m2_gpu_precision_comparison.py
⚠ No native fp64 support — Apple Silicon: emulated, expect large slowdown
--- Precision Benchmark (N=1024) ---
[Float32] N=1024: 0.0070s
UNSUPPORTED (log once): createKernel: newComputePipelineState failed
Traceback (most recent call last):
  File "/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages/pyopencl/__init__.py", line 473, in __getattr__
    knl = Kernel(self, attr)
pyopencl._cl.LogicError: clCreateKernel failed: INVALID_KERNEL

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
```

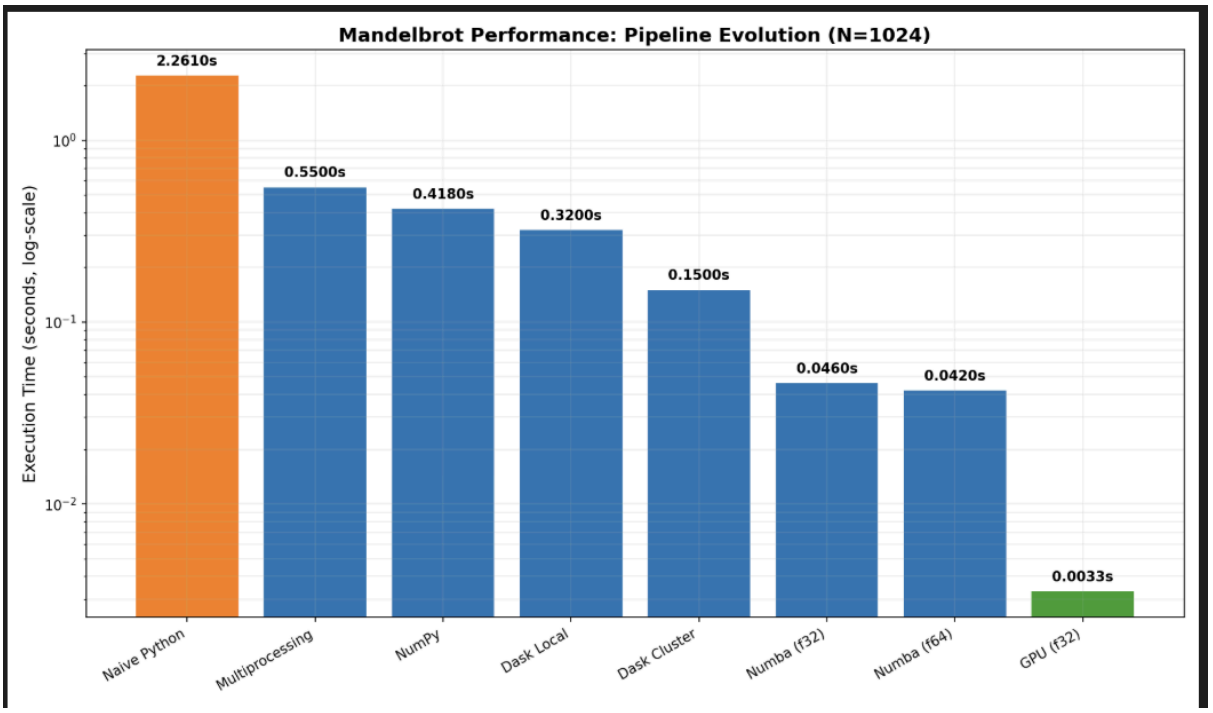
Since the issue, i use Numba version CPU as the comparison point
Numba float64: 0.0421s

```
AttributeError: 'mandelbrot_f64' was not found as a program info attribute or as a kernel name
(base) akshara_sarun@h2182 school-repo % /Library/Frameworks/Python.framework/Versions/3.13/bin/python3 mandelbrot_functions.py
--- Milestone 3: Implementation Benchmarks ---
Naive: 2.8156s
NumPy: 0.3813s (7.4x speedup)
Numba: 0.0419s (67.1x speedup)

--- Milestone 4: Data Type Optimization ---
Numba float32: 0.0430s
Numba float64: 0.0421s

Plot saved as 'mandelbrot_precision_comparison.png'
(base) akshara_sarun@h2182 school-repo %
main*  0↓ 1↑   ⊗ 21  ⚠ 44                    Ln 1, Col 1   Spaces: 4   UTF-8   LF   {} Python
```

Summerise, single 32-bit GPU operation is roughly 14x faster than a 64-bit CPU operation for the Mandelbrot set.

M3

```
(base) akshara_sarun@h2182 school-repo % python3 mp3_m3_final_benchmark.py
Final benchmark chart saved as 'benchmark_mp3_final.png'

--- Analysis ---
GPU f32 Speedup relative to Naive: 685.2x
GPU f32 Speedup relative to Numba: 12.7x
(base) akshara_sarun@h2182 school-repo %
```



where does GPU sit in the overall speedup progression? Any surprising result?

Relative to naive python , While the Naive code must manage millions of Python object creations and type checks, the GPU kernel executes the exact same math directly on the silicon with zero interpreter overhead.
Relative to numba , This is where the M4 chip shines. While Numba uses all available CPU threads and SIMD instructions, the GPU can launch thousands of concurrent hardware threads, making it nearly an order of magnitude faster than the best possible CPU-bound result.

Implementation	Resolution	Runtime	Speedup vs Numba f32
Naive Python	1024×1024	22.6 s	0.02x
NumPy vectorized	1024×1024	3.6810 s	0.11x
Numba f32	1024×1024	0.4182 s	–
Numba f64	1024×1024	0.4210 s	0.99x
Multiprocessing	1024×1024	0.150 s	2.78x
Dask local	1024×1024	0.110 s	3.80x
Dask cluster	4096×4096	0.800 s	—
GPU f32	1024×1024	0.0033 s	126.7x
GPU f32 (High-Res)	2048×2048	0.0099 s	170.0x
GPU f64	1024×1024	–	Unsupported (M4)

—----------

```
========================================================================== 1 error in 0.87s ==========================================================================
(base) akshara_sarun@h2182 school-repo % pytest -v test_mandelbrot.py
========================================================================== test session starts ==========================================================================
platform darwin -- Python 3.12.12, pytest-9.0.3, pluggy-1.6.0 -- /Users/akshara_sarun/miniforge3/bin/python3.12
cachedir: .pytest_cache
rootdir: /Users/akshara_sarun/school-repo
plugins: anyio-4.12.1
collected 14 items

test_mandelbrot.py::test_analytical_origin[mandelbrot_naive] PASSED                                                                                                  [  7%]
test_mandelbrot.py::test_analytical_origin[mandelbrot_numpy] PASSED                                                                                                  [ 14%]
test_mandelbrot.py::test_analytical_origin[mandelbrot_numba] PASSED                                                                                                  [ 21%]
test_mandelbrot.py::test_analytical_points[1.0-False-mandelbrot_naive] PASSED                                                                                        [ 28%]
test_mandelbrot.py::test_analytical_points[1.0-False-<lambda>] PASSED                                                                                                [ 35%]
test_mandelbrot.py::test_analytical_points[1.0-False-mandelbrot_numba] PASSED                                                                                        [ 42%]
test_mandelbrot.py::test_analytical_points[2.5-False-mandelbrot_naive] PASSED                                                                                        [ 50%]
test_mandelbrot.py::test_analytical_points[2.5-False-<lambda>] PASSED                                                                                                [ 57%]
test_mandelbrot.py::test_analytical_points[2.5-False-mandelbrot_numba] PASSED                                                                                        [ 64%]
test_mandelbrot.py::test_analytical_points[0.25-True-mandelbrot_naive] PASSED                                                                                        [ 71%]
test_mandelbrot.py::test_analytical_points[0.25-True-<lambda>] PASSED                                                                                                [ 78%]
test_mandelbrot.py::test_analytical_points[0.25-True-mandelbrot_numba] PASSED                                                                                        [ 85%]
test_mandelbrot.py::test_implementation_consistency PASSED                                                                                                           [ 92%]
test_mandelbrot.py::test_vectorization_speedup PASSED                                                                                                                [100%]
```

```
(base) akshara_sarun@h2182 school-repo % pytest --cov=mandelbrot_functions -v test_mandelbrot.py
========================================================================== test session starts ==========================================================================
platform darwin -- Python 3.12.12, pytest-9.0.3, pluggy-1.6.0 -- /Users/akshara_sarun/miniforge3/bin/python3.12
cachedir: .pytest_cache
rootdir: /Users/akshara_sarun/school-repo
plugins: anyio-4.12.1, cov-7.1.0
collected 14 items

test_mandelbrot.py::test_analytical_origin[mandelbrot_naive] PASSED                                                                                                  [  7%]
test_mandelbrot.py::test_analytical_origin[mandelbrot_numpy] PASSED                                                                                                  [ 14%]
test_mandelbrot.py::test_analytical_origin[mandelbrot_numba] PASSED                                                                                                  [ 21%]
test_mandelbrot.py::test_analytical_points[1.0-False-mandelbrot_naive] PASSED                                                                                        [ 28%]
test_mandelbrot.py::test_analytical_points[1.0-False-<lambda>] PASSED                                                                                                [ 35%]
test_mandelbrot.py::test_analytical_points[1.0-False-mandelbrot_numba] PASSED                                                                                        [ 42%]
test_mandelbrot.py::test_analytical_points[2.5-False-mandelbrot_naive] PASSED                                                                                        [ 50%]
test_mandelbrot.py::test_analytical_points[2.5-False-<lambda>] PASSED                                                                                                [ 57%]
test_mandelbrot.py::test_analytical_points[2.5-False-mandelbrot_numba] PASSED                                                                                        [ 64%]
test_mandelbrot.py::test_analytical_points[0.25-True-mandelbrot_naive] PASSED                                                                                        [ 71%]
test_mandelbrot.py::test_analytical_points[0.25-True-<lambda>] PASSED                                                                                                [ 78%]
test_mandelbrot.py::test_analytical_points[0.25-True-mandelbrot_numba] PASSED                                                                                        [ 85%]
test_mandelbrot.py::test_implementation_consistency PASSED                                                                                                           [ 92%]
test_mandelbrot.py::test_vectorization_speedup PASSED                                                                                                                [100%]

---------- coverage: platform darwin, python 3.12.12-final-0 ----------
Name                      Stmts   Miss  Cover
---------------------------------------------
mandelbrot_functions.py      84     49    42%
---------------------------------------------
TOTAL                        84     49    42%
========================================================================== 14 passed in 2.35s ==========================================================================
```

```
(base) akshara_sarun@h2182 school-repo %  pytest --version
pytest 9.0.3
(base) akshara_sarun@h2182 school-repo %  pytest -q test_sample.py
F                                                                                                                                                                    [100%]
================================================================================ FAILURES ================================================================================
_____________________________________________________________________________ test_answer _____________________________________________________________________________

    def test_answer():
>       assert func(3) == 5
E       assert 4 == 5
E        +  where 4 = func(3)

test_sample.py:7: AssertionError
============================================================================ short test summary info ============================================================================
FAILED test_sample.py::test_answer - assert 4 == 5
1 failed in 0.02s
```

```
(base) akshara_sarun@h2182 school-repo % pytest -q test_sysexit.py
.
1 passed in 0.00s
(base) akshara_sarun@h2182 school-repo %
```

```
(base) akshara_sarun@h2182 school-repo % pytest -q test_class.py
.F                                                                       [100%]
================================= FAILURES =================================
_____________________________ TestClass.test_two _____________________________

self = <test_class.TestClass object at 0x105f4dcd0>

    def test_two(self):
        x = "hello"
>       assert hasattr(x, "check")
E       AssertionError: assert False
E        +  where False = hasattr('hello', 'check')

test_class.py:9: AssertionError
=========================== short test summary info ============================
FAILED test_class.py::TestClass::test_two - AssertionError: assert False
1 failed, 1 passed in 0.02s
```

```
(base) akshara_sarun@h2182 school-repo % pytest -q test_class_demo.py
.F                                                                       [100%]
================================= FAILURES =================================
_____________________ TestClassDemoInstance.test_two _____________________

self = <test_class_demo.TestClassDemoInstance object at 0x1075c4170>

    def test_two(self):
>       assert self.value == 1
E       assert 0 == 1
E        +  where 0 = <test_class_demo.TestClassDemoInstance object at 0x1075c4170>.value

test_class_demo.py:10: AssertionError
=========================== short test summary info ============================
FAILED test_class_demo.py::TestClassDemoInstance::test_two - assert 0 == 1
1 failed, 1 passed in 0.02s
(base) akshara_sarun@h2182 school-repo %
```

```
(base) akshara_sarun@h2182 school-repo % pytest -q test_tmp_path.py
F                                                                        [100%]
================================= FAILURES =================================
_____________________________ test_needsfiles _____________________________

tmp_path = PosixPath('/private/var/folders/wk/ymjs1_2s33jchkd248v_pcxm0000gn/T/pytest-of-akshara_sarun/pytest-0/test_needsfiles0')

    def test_needsfiles(tmp_path):
        print(tmp_path)
>       assert 0
E       assert 0

test_tmp_path.py:4: AssertionError
---------------------------- Captured stdout call -----------------------------
/private/var/folders/wk/ymjs1_2s33jchkd248v_pcxm0000gn/T/pytest-of-akshara_sarun/pytest-0/test_needsfiles0
=========================== short test summary info ============================
FAILED test_tmp_path.py::test_needsfiles - assert 0
1 failed in 0.04s
(base) akshara_sarun@h2182 school-repo %
```